TRADED RISK MODEL DEVELOPMENT · OCTOBER 2026

Context engineering for AI-assisted model development



What the agent needs to know, where it lives in VS Code, and how to keep the window clean, worked on an FRTB SA example

VS Code · GitHub Copilot · FRTB SA (MAR21)

WHY CONTEXT

The agent only knows what is in its window

Each request is assembled from seven sources. Anything outside them does not exist for the model.

More is not better: in Chroma's 2025 test of 18 models, reliability fell as input grew.

The aim is the smallest set of high-signal tokens that makes the right output likely (Anthropic, 2025).

ONE REQUEST = ONE CONTEXT WINDOW

System instructions	built in
Customizations	you, once per repo
Your message	you, every request
Conversation history	grows every turn
Implicit context	active file, selection
Explicit references	you, via #file
Tool outputs	grows as the agent works

When it fills, VS Code compacts older turns into a summary, and detail can be lost.

RUNNING EXAMPLE

Adding GIRR delta to our FRTB SA engine

Little code, dense rules: every step hangs on a parameter or a convention the agent has to get exactly right.

Net sensitivities
s_k per currency × tenor
`MAR21.19`

→

Weighting
WS_k = RW_k · s_k
`MAR21.42–44`

→

Within currency
K_b = √(ΣΣ ρ·WS·WS), floored at 0
`MAR21.4(4), .46`

→

Across currencies
γ = 50%; alternative S_b if the sum < 0
`MAR21.4(5), .50`

→

Three scenarios
low · medium · high; max on the SBM total
`MAR21.6–7`

WHAT WE TYPE

```
Implement GIRR delta capital (MAR21) in
src/frtb/sa/girr.py
```

WHAT IT MUST ALSO KNOW

Which rule version · our elections · sensitivity units · the data schema · what counts as validated

WITHOUT CONTEXT

Five ways the agent gets GIRR delta wrong

What the agent does	Likely cause	Context that prevents it
Uses the 2016 GIRR weights (2.4% at 3M) instead of MAR21.42 (1.7%)	Training data holds both rule versions	Weights only from params/girr_delta.yaml, versioned and signed off
Mixes PV01 per bp with MAR21.19's ΔV ÷ 0.0001: capital off by 10,000×	Units are invisible in the code	Units stated in AGENTS.md; a one-swap unit test
Returns medium only, or takes the max per risk class	MAR21.6–7 order is easy to shortcut	Rule: return all three scenarios; max only on the SBM total
Skips the K_b floor and the alternative S_b (MAR21.4)	Assumes they never bind; both can	Independent golden cases: a butterfly, offsetting books
Applies the √2 divisor (MAR21.44) everywhere, or nowhere	It is a bank election, not a rule	Our election written into the scoped instructions

WHERE IT LIVES

Each piece of knowledge has a home in VS Code

LAYER	FILE IN THE REPO	LOADS	IN OUR EXAMPLE
Always-on	`AGENTS.md` or `.github/copilot-instructions.md`	Every request	Stack, test command, units, data rules
Scoped	`.github/instructions/frtb-sa.instructions.md`	applyTo match, or the task fits its description	SBM rules, our elections, MAR21 ¶ map
Workflow	`.github/skills/explain-girr-move/SKILL.md, waterfall.py, report.md`	A capital explain, or /explain-girr-move	Checklist, waterfall script, report template
Role	`.github/agents/mar21-reviewer.agent.md`	You pick it, or another agent delegates	Read-only review against docs/reg/
Per request	`#file:params/girr_delta.yaml + the failing test output`	You attach it	Exact inputs for this one task

Prompt files (.prompt.md) run only in the Local harness; for workflows you reuse across Copilot clients, write a skill.

IN THE REPO

Two short files carry most of the context

AGENTS.md every request

```
# FRTB SA capital engine
Python 3.12, numpy, pytest. Run: make test
Sensitivities per MAR21.19:
  s_k = ΔV(+1bp) / 0.0001, reporting ccy EUR
Methodology, golden cases: docs/methodology/
Rule texts: docs/reg/ (MAR21 extracts)
Parameters: params/*.yaml (signed off)
Test data: synthetic only, in tests/data/
Ask before changing src/frtb/api.py signatures
```

Left out on purpose

Style rules the linter already enforces, and anything the agent can read from the code itself.

.github/instructions/frtb-sa.instructions.md on match

```
---
description: Rules for FRTB SA (SBM) capital code
applyTo: "src/frtb/sa/**,tests/**"
---
- Load every RW, ρ and γ from params/*.yaml;
  never type one. Why: GIRR weights changed
  2016→2019; training data holds both.
- Return low, medium and high scenarios; take
  the max only on the SBM total (MAR21.7).
- Floor the √ argument of K_b at 0; if the
  across-bucket sum < 0, use the alternative
  S_b (MAR21.4). Both can bind: see G4, G5.
- GIRR √2 divisor (MAR21.44): not elected.
- Cite the MAR21 ¶ in every docstring.
- Done = make test green, incl. tests/golden/.
```

REUSABLE WORKFLOW

A skill packages the workflow, loaded on demand

`.github/skills/explain-girr-move/SKILL.md` on demand

```
---
name: explain-girr-move
description: Explain a move in FRTB SA GIRR
  delta capital between two dates. Use for daily
  or month-end capital explains.
---
1. Put both dates' sensitivities and official
   results in [inputs.json](./inputs.json) format.
2. Check what changed besides positions: params,
   units (MAR21.19), curve and tenor mapping.
3. Run [waterfall.py](./scripts/waterfall.py),
   one currency at a time. No hand arithmetic.
4. A date that doesn't reproduce: stop and fix
   it first. Show a scenario switch separately.
5. Write it up with [report.md](./report.md); name
   the tenors behind each move.
```

HOW IT LOADS

1 · Always in context

Name and description, one line per skill. The description must say when to use it.

2 · On a capital explain, or /explain-girr-move

The SKILL.md body: five steps, one checklist

3 · Only when a step links to it

inputs.json, waterfall.py, report.md

Sample run on the bundled inputs.json: a EUR 2y hedge unwind adds EUR 0.46m. Like-for-like in low: EUR +0.35m, USD +0.03m. Switch from low to high: +0.08m.

DURING THE TASK

Run each task in phases with a clean window

1 · Explore

`subagent, read-only`

Reads widely in its own window; only a summary comes back.

2 · Plan

`Plan agent → file`

Save plans/girr-delta.md; check scenarios, floors, election.

3 · Implement

`new chat + plan file`

Golden tests first, then code, then make test.

4 · Review

`fresh chat, reviewer`

Checks the diff against the plan and docs/reg/.

Throughout the task

Context meter

In the chat input. Compact before it fills.

`/compact`

Give it a focus: keep decisions, drop logs.

`/fork`

Try an alternative without losing the thread.

Checkpoints

Rewind file edits; terminal effects stay.

BEST PRACTICES AND PITFALLS

What works, and what to avoid

Do

Rules on evidence. Add one after the agent repeats a mistake, and give the reason.

Point, don't paste. Reference params/ and docs/reg/ so the agent reads the source.

Executable "done". Tests and golden results the agent runs itself.

Short always-on files. Scope the rest with applyTo and skills.

Context is code. Version it and review it in pull requests.

Check what loaded. References in the reply; Agent Debug Logs.

Avoid

Context dumping. Whole PDFs or 50k-row files; give a schema and 20 rows.

Marathon chats. Failed attempts keep steering; compaction drops detail.

Conflicting files. Instruction sources add up; nothing takes precedence.

Recalled regulation. And shared skills nobody has reviewed.

Every tool switched on. Worse tool choice; fetched pages can inject instructions.

Sensitive data in prompts. No positions, client names or production paths.

NEXT WEEK

Start with three files and one habit

`AGENTS.md`

One page: stack, test command, units, data rules.

`frtb-sa.instructions.md`

Scoped rules and our elections, each with its reason.

`params/ + tests/golden/`

Ground truth the agent reads, and must pass.

The habit: one task per chat. Explore, plan to a file, implement in a fresh chat, and let the tests decide when it’s done.

REFERENCE

Sources and what is illustrative

Sources

VS Code documentation, accessed 24 Oct 2026: Context, Custom instructions, Prompt files, Agent skills, Subagents, Best practices, Context engineering guide.

Anthropic Engineering, Effective context engineering for AI agents (Sep 2025).

Chroma, Context Rot: How Increasing Input Tokens Impacts LLM Performance (2025).

BCBS, MAR21 (in force 1 Jan 2023, last updated 23 Mar 2026): 21.4, 21.6–21.7, 21.19, 21.42–21.50.

2016 GIRR weights (BCBS d352) as tabulated in 1 PLUS i, FRTB notes (Jan 2019).

Computed for this deck: the smallest eigenvalue of MAR21.46 Table 2 (about −0.165), the golden cases and the skill's sample run.

Illustrative, not prescriptive

Repository layout, file and skill names, and make test.

The √2 election, EUR reporting and the synthetic desk in the skill's sample.

The failure modes are expected behaviour, not logged incidents. The 2016 weights do appear in at least one publicly shared agent skill.